

Getting Started

Getting Started · docs/latest · generated 2026-08-15

This guide covers the three ways to work with StarUI: consuming the packages in **your own application**, running the **demo apps**, and developing the **platform itself**.

1. Consume the packages in your app

The packages install as ordinary npm packages under their real names. From a registry (Artifactory) that hosts them:

```
npm install @wellsfargo-starui/grid @wellsfargo-starui/react \
  @wellsfargo-starui/design-system
```

Without a registry, install the packed tarballs directly — `npm run pack:npm` in the platform repo emits one tarball per package under `dist-npm/`:

```
npm install /path/to/dist-npm/wellsfargo-starui-grid-0.1.0.tgz
```

The packed tarballs depend on each other by version. Installing from files with no registry needs an `overrides` block naming the whole set — see the generated `apps/tarball/*/package.json` for the working shape.

Your app owns the framework peers:

```
npm install react react-dom ag-grid-community ag-grid-enterprise \
  ag-grid-react @tanstack/react-query tailwindcss
```

A minimal grid host

The `apps/source/basic` app is the canonical tutorial — a complete bond blotter in a few files. The essential wiring:

```
import {
  MarketsGrid,
  createMarketsGridLocalStorageStorage,
} from '@wellsfargo-starui/grid';
import { applyTheme, getTheme } from '@wellsfargo-starui/design-system';
import '@wellsfargo-starui/design-system/styles.css';
import '@wellsfargo-starui/grid/styles.css';

const storage = createMarketsGridLocalStorageStorage();

export function App() {
  return (
    <MarketsGrid
      gridId="bond-blotter-v1"          // stable id – keys profile persistence
      rowData={rows}
      columnDefs={columnDefs}
      defaultColDef={defaultColDef}
      rowIdField="id"
      storage={storage}                // where profiles + grid state persist
      showFiltersToolbar
      showFormattingToolbar
      showEditingToolbar
    />
  );
}
```

What you get out of the box: profile persistence (columns, filters, formats survive reload), the filters / formatting / editing toolbars, the column customizer, and full dark/light theming.

Theming

Themes are design-system tokens switched by one attribute:

```
import { applyTheme, getTheme } from '@wellsfargo-starui/design-system';

applyTheme({ theme: 'dark' });          // flips data-theme on <html>
const { theme } = getTheme();           // 'dark' | 'light'
```

Never hardcode colors — consume `--bn-*` / `--fi-*` CSS variables or the semantic exports from `@wellsfargo-starui/design-system/tokens/semantic`. Every surface must render correctly under both `data-theme="dark"` and `data-theme="light"`.

UI primitives

Build app chrome from `@wellsfargo-starui/react` (shadcn/Radix primitives pre-wired to the tokens) — not native `<input>` / `<select>` / `<textarea>`:

```
import { Button, Tooltip, TooltipTrigger, TooltipContent } from '@wellsfargo-starui/react';
```

Live data — the whole platform in 27 lines

The north-star app is `apps/source/hello-blotter`: a live 20,000-row SSRM blotter in one 27-line file and two starui import specifiers. This is the recommended shape for a new app — `createStarui()` boots the platform (SharedWorker data hub, provider catalog, storage, identity) and `<StarGrid>` renders the grid, inferring its mode from the provider:

```
import { createRoot } from 'react-dom/client';
import { createStarui } from '@wellsfargo-starui/react/data/runtime';
import { StarGrid } from '@wellsfargo-starui/grid/widgets';
import './index.css';

const starui = createStarui({
  appId: 'HelloBlotter',
  userId: 'demo',
  providers: [{
    providerId: 'dp-hello-positions', name: 'Positions (live)',
    providerType: 'stomp-ssrm', userId: 'demo',
    config: {
      providerType: 'stomp-ssrm',
      websocketUrl: 'ws://localhost:8081',
      listenerTopic: '/snapshot/positions/trd1',
      requestMessage: '/snapshot/positions/trd1/1000/10',
      snapshotEndToken: 'Success',
      keyColumn: 'positionId', publishWindowMs: 200,
    },
  ]},
});

createRoot(document.getElementById('root')).render(
  <starui.Provider>
    <StarGrid gridId="hello-blotter" providerId="dp-hello-positions" title="Positions"
fullBleed />
  </starui.Provider>,
);
```

with a two-line stylesheet (`index.css`):

```
@import '@wellsfargo-starui/design-system/css';
@import '@wellsfargo-starui/grid/styles.css';
```

What the 27 lines buy: one upstream STOMP connection shared by every window, server-side row model paging from the SharedWorker's query plane, columns inferred from the feed (no `columnDefs`), the full customizer + profile persistence, dark/light theming, and workspace-save flushing under OpenFin. Notes:

- `providers` seeds the catalog **create-if-missing** — the `providerId` must be deterministic (random ids would re-seed a new row every launch), and later edits in the Data Provider Editor survive reloads.
- `<StarGrid>` infers its mode: a named SSRM provider → SSRM container; a CSRM provider → CSRM container; `rowData` → static grid; neither → a container whose provider is picked at runtime in the

customizer.

- The full provider-config field reference is [Provider Config Reference](#) →; the AppData key/value layer is [AppData](#) →.

Run it against live data (from this repo):

```
npm run app -- hello-blotter      # starts the STOMP fixture feed (:8081) + the app (:5177)
```

Open <http://localhost:5177> — the blotter fills with a 20k-row snapshot and ticks live updates. The feed is `apps/source/stomp-view-server` (synthetic fixed-income positions over STOMP-with-WebSocket); the provider's `requestMessage` `/snapshot/positions/trd1/1000/10` asks for client id `trd1` at 1,000 row-updates/sec in batches of 10.

2. Run the demo apps

The demos live in this repo under `apps/` — **their own npm install root**, separate from the package workspaces. One command from the platform repo root builds the packages, packs them, and installs `apps/` for both consumption tracks (`source/` and the generated `tarball/` twins):

```
npm run setup:apps
```

Equivalent, step by step:

```
npm install && npm run build
npm run pack:npm      # only needed for the tarball track

cd apps
npm install
npm run setup:tarball # vendor, regenerate, install the tarball twins
npm run typecheck && npm run build # both consumption tracks
```

The easiest way to run any app is the root-level runner — it knows when an app needs the STOMP fixture broker and starts it for you, and it drives the OpenFin launcher when asked:

```
npm run app           # list apps + ports
npm run app -- basic   # one app, source track
npm run app -- hello-blotter # broker starts automatically
npm run app -- star-demo --openfin # dev server, then OpenFin platform
npm run app -- markets-grid-lab --tarball # the generated twin (:6300)
```

(`cd apps/source/<app> && npm run dev` still works for any single app.)

APP	PORT	PURPOSE
hello-blotter	5177	the north star — live SSRM blotter in 27 lines (<code>createStarui</code> + <code><StarGrid></code>)
star-demo	5175	OpenFin workspace demo; primary e2e target
star-demo-ssrm	5176	star-demo's SSRM twin (<code>stomp-ssrm</code> provider)
markets-grid-lab	5300	MarketsGrid editing / profiles lab
markets-grid-ssrm-lab	5320	SSRM feature lab (mock-ssrm provider)
design-system	5310	design-system showcase
stomp-marketsgrid-minimal	5213	smallest STOMP → grid path
basic	5194	tutorial — minimal grid host
datapreview-editor	5193	tutorial — data-provider editor
stomp-view-server	8081	STOMP fixture server (not a UI)

Each UI app also has a generated **tarball twin** (same code, consuming vendored tarballs, port +1000). Regenerate the twins after package changes:

```
npm run pack:npm && cd apps && npm run setup:tarball # vendor, regenerate, install
```

3. Develop the platform

```
npm install          # workspaces: packages/* only
npm run build        # turbo build + consumer-tsconfig regeneration
npm run typecheck    # build, then turbo typecheck
npm test            # Vitest across packages/
npm run lint:all     # eslint + cycles + token rules + RTL check
```

Before shipping a change:

```
npx turbo typecheck build test # the green bar
npm run test:coverage          # per-file coverage run
npm run check:coverage         # 70%-per-file gate
```

Conventions that will save you a review round-trip:

- **Coverage is per file** — 70% on lines, statements, functions and branches, with `all: true`, so a new untested file fails the gate at 0%.
- React components are tested with **React Testing Library** (`npm run check:rtl` enforces it).

- Filenames match the case of the primary export; folders are kebab-case.
- Update `docs/current-features.md` in the same change as any feature add/update/remove.
- Never `npm ci`, never `pnpm`/`yarn`, no `--legacy-peer-deps` — plain `npm install` must always resolve cleanly.

The architecture and layer rules are in [Architecture](#) →.